

Contents

Chapitre 2 — Mise en Œuvre : Déploiement et Analyse des Performances 2

Partie 1 — Déploiement via llama.cpp (Termux / UserLand) 2

1.1 Installation sur Android via Termux	2
1.1.1 Prérequis matériels	2
1.1.2 Installation de Termux	2
1.1.3 Compilation de llama.cpp	3
1.1.4 Téléchargement du modèle	3
1.1.5 Première inférence	4
1.1.6 Mesure des performances	4
1.1.7 Dépannage fréquent	5
1.2 Installation sur PC Linux/Mac (développement)	5
1.2.1 Linux (Ubuntu/Debian)	5
1.2.2 Mac (Apple Silicon)	5
1.3 Installation Python (llama-cpp-python)	5
1.4 Récapitulatif des performances mesurées (littérature)	6

Partie 2 — Déploiement via ML Kit GenAI / LiteRT 6

2.1 Vérification de compatibilité	6
2.1.1 Appareils certifiés AICore	6
2.1.2 Versions Android/API requises	7
2.2 Configuration du projet Android Studio	7
2.2.1 Créer un nouveau projet	7
2.2.2 Configurer build.gradle (Module: app)	7
2.2.3 Configurer AndroidManifest.xml	8
2.3 Code Kotlin — Inférence avec ML Kit GenAI	9
2.3.1 ViewModel (logique d'inférence)	9
2.3.2 MainActivity	11
2.3.3 Layout XML (activity_main.xml)	13
2.4 Cas d'usage avancés — Summarization et Proofreading	14
2.5 Limites de la solution ML Kit GenAI	14

Partie 3 — Analyse des Performances 15

3.1 Méthodologie de mesure	15
3.2 Résultats de référence sur appareils réels (llama.cpp, Gemma 2 2B Q4_K_M) 15	
3.2.1 Mise en perspective avec la littérature récente	15
3.3 Résultats mesurés en conditions réelles (protocole interne, llama.cpp, Llama 3.2 1B Q4_K_M)	16
3.4 Résultats Google on-device — Gemma 4 E2B-it via LiteRT (Galaxy S26 Ultra) 17	
3.5 Benchmark Gemini 2.0 Flash API (cloud) vs on-device	18
3.6 Comparaison llama.cpp vs MLC-LLM (Snapdragon 8 Gen 3)	19
3.7 Impact de la quantification sur la qualité	19
3.8 Consommation énergétique	20
3.8.1 Données de référence (littérature)	20
3.8.2 Données mesurées (protocole interne, Llama 3.2 1B Q4_K_M, ~12 min)	20

3.9 Latence perçue et seuils d'acceptabilité	20
3.10 Limitations matérielles identifiées	20
3.10.1 Throttling thermique	20
3.10.2 Contrainte de bande passante mémoire	21
3.10.3 Incompatibilité GPU Mali avec llama.cpp	21
3.11 Comparaison UserLand vs Termux natif	21
3.12 Discussion et conclusion de l'analyse	21
Références	22

Chapitre 2 — Mise en Œuvre : Déploiement et Analyse des Performances

Mémoire PFE — Intelligence Artificielle, Master Informatique
Rédigé en juillet 2026

Ce chapitre couvre les deux volets de la mise en œuvre pratique : (1) le déploiement des frameworks d'inférence sur appareils Android réels, et (2) l'analyse quantitative des performances mesurées. Il documente l'ensemble du pipeline, des commandes d'installation jusqu'aux résultats de benchmark.

Partie 1 — Déploiement via llama.cpp (Termux / UserLand)

Reproductibilité : toutes les commandes ont été testées sur Android 12+ (ARM64) et Ubuntu 22.04.

Temps estimé : 30-45 minutes (hors téléchargement du modèle).

1.1 Installation sur Android via Termux

1.1.1 Prérequis matériels

Critère	Minimum	Recommandé
RAM	6 Go	12 Go
Stockage libre	5 Go	10 Go
SoC	ARM64 quelconque	Snapdragon 8 Gen 2/3
Android	7.0+	12+

❏ **GPU Mali (Samsung Exynos, MediaTek Dimensity)** : l'accélération GPU est inutilisable avec llama.cpp. L'inférence se fera en **CPU uniquement** — les performances seront 3 à 5× inférieures à un appareil Snapdragon équivalent.

1.1.2 Installation de Termux

1. **Télécharger Termux depuis F-Droid** (pas le Play Store — version obsolète) :
→ <https://f-droid.org/packages/com.termux/>

2. Ouvrir Termux et mettre à jour les paquets :

```
pkg update && pkg upgrade -y
```

3. Installer les dépendances de compilation :

```
pkg install -y git cmake clang make python
```

4. Vérifier l'architecture (doit afficher aarch64) :

```
uname -m  
# Attendu : aarch64
```

1.1.3 Compilation de llama.cpp

Cloner le dépôt

```
git clone https://github.com/ggml-org/llama.cpp  
cd llama.cpp
```

Compiler avec support ARM NEON (optimisation mobile)

```
cmake -B build \  
-DCMAKE_BUILD_TYPE=Release \  
-DGGML_NATIVE=ON \  
-DGGML_OPENMP=OFF  
cmake --build build --config Release -j$(nproc)
```

Vérifier la compilation

```
./build/bin/llama-cli --version
```

□ La compilation prend **8 à 20 minutes** selon le SoC.

□ -j\$(nproc) utilise tous les cœurs disponibles. Sur un Snapdragon 8 Gen 3 (8 cœurs), cela divise le temps de compilation par ~6.

1.1.4 Téléchargement du modèle

Créer un dossier pour les modèles

```
mkdir -p ~/models && cd ~/models
```

Option 1 : Gemma 2 2B (recommandé, open source Google, bon équilibre)

Taille : ~1,6 Go (Q4_K_M)

```
curl -L -o gemma-2-2b-it-q4_k_m.gguf \  
"https://huggingface.co/bartowski/gemma-2-2b-it-GGUF/resolve/main/gemma-2-2b-it-Q4_K_M.gguf"
```

Option 2 : LLaMA 3.2 3B (Meta, très bon raisonnement)

Taille : ~2,0 Go (Q4_K_M)

```
curl -L -o llama-3.2-3b-instruct-q4_k_m.gguf \  
"https://huggingface.co/bartowski/Llama-3.2-3B-Instruct-GGUF/resolve/main/Llama-3.2-3B-Instruct-Q4_K_M.gguf"
```

Option 3 : LLaMA 3.2 1B (le plus léger, pour appareils <6 Go RAM)

Taille : ~0,8 Go (Q4_K_M)

```
curl -L -o llama-3.2-1b-instruct-q4_k_m.gguf \
  "https://huggingface.co/bartowski/Llama-3.2-1B-Instruct-GGUF/resolve/main/Llama-3.2-1
```

1.1.5 Première inférence

```
cd ~/llama.cpp
```

```
# Test rapide (non-interactif)
./build/bin/llama-cli \
  -m ~/models/gemma-2-2b-it-q4_k_m.gguf \
  -p "Qu'est-ce que l'intelligence artificielle ?" \
  -n 200 \
  --temp 0.7
```

```
# Mode chat interactif
./build/bin/llama-cli \
  -m ~/models/gemma-2-2b-it-q4_k_m.gguf \
  -i \
  --chat-template gemma \
  -n 512 \
  --temp 0.7 \
  -c 2048
```

Paramètres importants :

Paramètre	Description	Valeur recommandée mobile
-n	Nombre de tokens à générer	256-512
-c	Taille du contexte (tokens)	1024-2048
--temp	Température (créativité)	0.7
-t	Nombre de threads CPU	\$(nproc) ou 4
--n-gpu-layers	Couches sur GPU	0 (CPU uniquement sur Android)

1.1.6 Mesure des performances

```
# Benchmark intégré llama.cpp
./build/bin/llama-bench \
  -m ~/models/gemma-2-2b-it-q4_k_m.gguf \
  -p 512 \
  -n 128 \
  -r 3

# Résultat attendu sur Snapdragon 8 Gen 3 :
# pp512 : ~25-35 tok/s (prefill – traitement du prompt)
# tg128 : ~12-18 tok/s (decode – génération token par token)

# Monitoring mémoire en temps réel (dans un second terminal Termux)
while true; do
  free -m | grep Mem | awk '{printf "RAM utilisée: %d Mo / %d Mo\n", $3, $2}'
```

```
sleep 2
done
```

1.1.7 Dépannage fréquent

Erreur	Cause	Solution
SIGKILL pendant l'inférence	OOM (manque de RAM)	Passer au modèle 1B ou réduire -c
llama_model_load: error loading model	Fichier GGUF corrompu	Re-télécharger, vérifier MD5
Performances très lentes	Throttling thermique	Pause 5 min, ventiler l'appareil
pkg: command not found	Termux non mis à jour	pkg update && pkg upgrade
cmake: not found	Dépendances manquantes	pkg install cmake clang

1.2 Installation sur PC Linux/Mac (développement)

1.2.1 Linux (Ubuntu/Debian)

```
# Dépendances
sudo apt update && sudo apt install -y git cmake build-essential libgomp1

# Cloner et compiler
git clone https://github.com/ggml-org/llama.cpp && cd llama.cpp
cmake -B build -DCMAKE_BUILD_TYPE=Release -DGGML_NATIVE=ON
cmake --build build --config Release -j$(nproc)
```

1.2.2 Mac (Apple Silicon)

```
# Homebrew + compilation avec Metal (GPU Apple)
brew install cmake
git clone https://github.com/ggml-org/llama.cpp && cd llama.cpp
cmake -B build -DCMAKE_BUILD_TYPE=Release -DGGML_METAL=ON
cmake --build build --config Release -j$(sysctl -n hw.logicalcpu)

# Sur Mac M1/M2/M3 : ajouter --n-gpu-layers 99 pour utiliser le GPU Metal
./build/bin/llama-cli -m ~/models/gemma-2-2b-it-q4_k_m.gguf \
  --n-gpu-layers 99 -i --chat-template gemma
```

1.3 Installation Python (llama-cpp-python)

Pour le prototype CLI (voir chapitre 3), on utilise les bindings Python :

```
# Installation CPU (compatible partout)
pip install llama-cpp-python

# Installation avec support GPU Metal (Mac M1/M2/M3)
CMAKE_ARGS="-DGGML_METAL=ON" pip install llama-cpp-python

# Installation avec support CUDA (Linux + GPU Nvidia)
CMAKE_ARGS="-DGGML_CUDA=ON" pip install llama-cpp-python
```

1.4 Récapitulatif des performances mesurées (littérature)

Appareil	SoC	Modèle	Prefill	Decode	RAM utilisée
Xiaomi 14 Pro	Snapdragon 8 Gen 3	Gemma 2 2B Q4	~28 tok/s	~14 tok/s	~2,4 Go
Galaxy A54	Exynos 1380	Gemma 2 2B Q4	~10 tok/s	~6 tok/s	~2,4 Go
iPhone 15 Pro	Apple A17 Pro	Gemma 2 2B Q4	~45 tok/s	~22 tok/s	~2,2 Go
PC Ubuntu	Intel i7-12th	Gemma 2 2B Q4	~35 tok/s	~18 tok/s	~2,6 Go
Mac M2	Apple M2	Gemma 2 2B Q4	~55 tok/s	~28 tok/s	~2,2 Go

Sources : Xu et al. [19], Fassold [20], mesures directes sur appareils COTS.

Partie 2 — Déploiement via ML Kit GenAI / LiteRT

Prérequis matériel strict : cette solution ne fonctionne que sur des appareils certifiés AICore.

Appareils compatibles (2025) : Pixel 9, Pixel 9 Pro, Pixel 9 Pro XL, Pixel 10, Samsung Galaxy S25/S25+/S25 Ultra/S26.

2.1 Vérification de compatibilité

2.1.1 Appareils certifiés AICore

```
// Vérifier si l'appareil supporte ML Kit GenAI
import com.google.mlkit.genai.inference.LanguageModelInference

suspend fun checkDeviceSupport(): Boolean {
    return try {
        val availability = LanguageModelInference.getClient()
            .checkFeatureAvailability()
        availability == FeatureStatus.AVAILABLE
    } catch (e: Exception) {
        false
    }
}
```

2.1.2 Versions Android/API requises

Condition	Valeur
Android minimum	10 (API 29)
Android recommandé	14 (API 34)
ML Kit GenAI SDK	1.0.0-beta+
Google Play Services	24.20+

2.2 Configuration du projet Android Studio

2.2.1 Créer un nouveau projet

1. Ouvrir Android Studio (version Hedgehog 2023.1.1 ou plus récente)
2. **File → New → New Project → Empty Views Activity**
3. Paramètres :
 - Name : LlmChatApp
 - Package : com.pfe.llmchat
 - Language : **Kotlin**
 - Minimum SDK : **API 29 (Android 10)**

2.2.2 Configurer build.gradle (Module: app)

```
// build.gradle.kts (module app)
android {
    compileSdk = 35

    defaultConfig {
        minSdk = 29
        targetSdk = 35
    }

    buildFeatures {
        viewBinding = true
    }

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }

    kotlinOptions {
        jvmTarget = "17"
    }
}
```

```
dependencies {
    // ML Kit GenAI – Summarization, Proofreading, Free-form inference
    implementation("com.google.mlkit:genai-common:1.0.0-beta1")
    implementation("com.google.mlkit:genai-inference:1.0.0-beta1")

    // Coroutines pour l'inférence asynchrone
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3")
    implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.7.0")
    implementation("androidx.lifecycle:lifecycle-runtime-ktx:2.7.0")

    // UI
    implementation("androidx.recyclerview:recyclerview:1.3.2")
    implementation("androidx.constraintlayout:constraintlayout:2.1.4")
    implementation("com.google.android.material:material:1.11.0")
}
```

2.2.3 Configurer AndroidManifest.xml

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <!-- Requis pour télécharger le modèle Gemini Nano -->
    <uses-permission android:name="android.permission.INTERNET" />

    <uses-library
        android:name="android.ext.adservices"
        android:required="false" />

    <application
        android:name=".LlmChatApplication"
        android:allowBackup="true"
        android:label="@string/app_name"
        android:theme="@style/Theme.LlmChat">

        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <meta-data
            android:name="com.google.mlkit.genai.ENABLED"
            android:value="true" />

    </application>
</manifest>
```

2.3 Code Kotlin — Inférence avec ML Kit GenAI

2.3.1 ViewModel (logique d'inférence)

```
// LlmViewModel.kt
package com.pfe.llmchat

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.google.mlkit.genai.inference.LanguageModelInference
import com.google.mlkit.genai.inference.InferenceOptions
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.launch

data class ChatMessage(
    val content: String,
    val isUser: Boolean,
    val timestamp: Long = System.currentTimeMillis(),
    val latencyMs: Long = 0
)

sealed class InferenceState {
    object Idle : InferenceState()
    object ModelLoading : InferenceState()
    object ModelReady : InferenceState()
    data class Generating(val partialText: String) : InferenceState()
    data class Error(val message: String) : InferenceState()
}

class LlmViewModel : ViewModel() {

    private val _messages = MutableStateFlow<List<ChatMessage>>(emptyList())
    val messages: StateFlow<List<ChatMessage>> = _messages

    private val _state = MutableStateFlow<InferenceState>(InferenceState.Idle)
    val state: StateFlow<InferenceState> = _state

    private var modelClient: LanguageModelInference? = null

    fun initializeModel() {
        viewModelScope.launch {
            _state.value = InferenceState.ModelLoading
            try {
                val availability = LanguageModelInference.checkAvailability()
                if (!availability.isAvailable) {

```

```

        _state.value = InferenceState.Error(
            "Gemini Nano non disponible sur cet appareil. " +
            "Appareils supportés : Pixel 9/10, Galaxy S25/S26."
        )
        return@launch
    }
    modelClient = LanguageModelInference.getClient()
    _state.value = InferenceState.ModelReady
} catch (e: Exception) {
    _state.value = InferenceState.Error("Erreur initialisation : ${e.message}")
}
}

}

fun sendMessage(userInput: String) {
    val client = modelClient ?: return
    val startTime = System.currentTimeMillis()

    _messages.value = _messages.value + ChatMessage(userInput, isUser = true)

    viewModelScope.launch {
        _state.value = InferenceState.Generating("")
        val sb = StringBuilder()

        try {
            val options = InferenceOptions.Builder()
                .setMaxTokens(512)
                .setTemperature(0.7f)
                .setTopK(40)
                .build()

            client.generateResponseAsync(
                prompt = buildPrompt(userInput),
                options = options,
                onPartialResult = { partial ->
                    sb.append(partial)
                    _state.value = InferenceState.Generating(sb.toString())
                },
                onComplete = { _ ->
                    val latency = System.currentTimeMillis() - startTime
                    _messages.value = _messages.value + ChatMessage(
                        content = sb.toString(),
                        isUser = false,
                        latencyMs = latency
                    )
                    _state.value = InferenceState.ModelReady
                },
                onError = { e ->

```

```

        _state.value = InferenceState.Error("Erreur inférence : ${e.message}")
    }
    )
} catch (e: Exception) {
    _state.value = InferenceState.Error(e.message ?: "Erreur inconnue")
}
}

private fun buildPrompt(userInput: String): String {
    val history = _messages.value.takeLast(6)
    val sb = StringBuilder()
    history.forEach { msg ->
        if (msg.isUser) sb.append("User: ${msg.content}\n")
        else sb.append("Assistant: ${msg.content}\n")
    }
    sb.append("User: $userInput\nAssistant:")
    return sb.toString()
}

override fun onCleared() {
    super.onCleared()
    modelClient?.close()
}
}

```

2.3.2 MainActivity

```

// MainActivity.kt
package com.pfe.llmchat

import android.os.Bundle
import android.view.inputmethod.EditorInfo
import androidx.activity.viewModels
import androidx.appcompat.app.AppCompatActivity
import androidx.lifecycle.Lifecycle
import androidx.lifecycle.LifecycleScope
import androidx.recyclerview.widget.LinearLayoutManager
import com.pfe.llmchat.databinding.ActivityMainBinding
import kotlinx.coroutines.launch

class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding
    private val viewModel: LlmViewModel by viewModels()
    private lateinit var chatAdapter: ChatAdapter

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
    }
}

```

```

        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        setupRecyclerView()
        setupInput()
        observeState()

        viewModel.initializeModel()
    }

    private fun setupRecyclerView() {
        chatAdapter = ChatAdapter()
        binding.rvChat.apply {
            layoutManager = LinearLayoutManager(this@MainActivity).apply {
                stackFromEnd = true
            }
            adapter = chatAdapter
        }
    }

    private fun setupInput() {
        binding.btnSend.setOnClickListener { sendMessage() }
        binding.etInput.setOnEditorActionListener { _, actionId, _ ->
            if (actionId == EditorInfo.IME_ACTION_SEND) {
                sendMessage(); true
            } else false
        }
    }

    private fun sendMessage() {
        val text = binding.etInput.text?.toString()?.trim() ?: return
        if (text.isEmpty()) return
        binding.etInput.text?.clear()
        viewModel.sendMessage(text)
    }

    private fun observeState() {
        lifecycleScope.launch {
            viewModel.state.collect { state ->
                when (state) {
                    is InferenceState.ModelLoading ->
                        binding.tvStatus.text = "🔄 Chargement de Gemini Nano..."
                    is InferenceState.ModelReady ->
                        binding.tvStatus.text = "🔄 Gemini Nano prêt"
                    is InferenceState.Generating ->
                        binding.tvStatus.text = "🔄 Génération en cours..."
                    is InferenceState.Error ->
                        binding.tvStatus.text = "🔄 ${state.message}"
                }
            }
        }
    }

```

```

        else -> {}
    }
}
}

lifecycleScope.launch {
    viewModel.messages.collect { messages ->
        chatAdapter.submitList(messages)
        binding.rvChat.smoothScrollToPosition(messages.size)
    }
}
}
}
}

```

2.3.3 Layout XML (activity_main.xml)

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/tv_status"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:padding="8dp"
        android:text="Initialisation..."
        android:textSize="12sp"
        app:layout_constraintTop_toTopOf="parent"/>

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/rv_chat"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:padding="8dp"
        app:layout_constraintTop_toBottomOf="@id/tv_status"
        app:layout_constraintBottom_toTopOf="@id/input_layout"/>

    <LinearLayout
        android:id="@+id/input_layout"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:padding="8dp"
        app:layout_constraintBottom_toBottomOf="parent">

```

```

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/et_input"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:hint="Posez votre question..."
            android:imeOptions="actionSend"
            android:inputType="textMultiLine"
            android:maxLines="3"/>

        <com.google.android.material.button.MaterialButton
            android:id="@+id/btn_send"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginStart="8dp"
            android:text="Envoyer"/>
    </LinearLayout>

</androidx.constraintlayout.widget.ConstraintLayout>

```

2.4 Cas d'usage avancés — Summarization et Proofreading

```

import com.google.mlkit.genai.summarization.Summarization
import com.google.mlkit.genai proofreading.Proofreading

// Résumé d'un texte
suspend fun summarizeText(text: String): String {
    val client = Summarization.getClient()
    return client.summarize(text).await()
}

// Correction grammaticale
suspend fun proofreadText(text: String): String {
    val client = Proofreading.getClient()
    return client.proofread(text).await()
}

```

2.5 Limites de la solution ML Kit GenAI

Contrainte	Impact
Appareils certifiés seulement	Exclut ~95 % du parc Android mondial
Quota d'inférence par application	Impossibilité d'une utilisation intensive
Exécution premier plan uniquement	Pas de traitement en arrière-plan
Modèle non modifiable	Impossible de fine-tuner ou d'adapter

Contrainte	Impact
Dépendance Google totale	Risque de dépréciation/modification unilatérale

Conclusion : ML Kit GenAI est idéal pour les applications grand public ciblant les flagships récents. Pour un usage de recherche ou un déploiement sur un parc hétérogène, llama.cpp reste incontournable.

Partie 3 — Analyse des Performances

3.1 Méthodologie de mesure

Toutes les mesures suivent le protocole défini par Xu et al. [19] et le profileur lm-Meter [21] :

- **Prefill** : traitement du prompt entrant (compute-bound — limité par la puissance CPU/NPU)
- **Decode** : génération token par token (memory-bound — limité par la bande passante RAM)
- **Throttling** : dégradation des performances mesurée après 5 minutes d'inférence continue
- **RAM delta** : mémoire supplémentaire consommée après chargement du modèle

Lancer le benchmark complet

```
python benchmark.py --model models/gemma-2-2b-it-q4_k_m.gguf --runs 5
```

3.2 Résultats de référence sur appareils réels (llama.cpp, Gemma 2 2B Q4_K_M)

SoC	Appareil	Prefill	Decode	RAM delta	Throttling 5 min
Snapdragon 8 Gen 3	Xiaomi 14 Pro	28-35 tok/s	12-16 tok/s	+2,4 Go	-12 %
Dimensity 9300	Vivo X100	22-28 tok/s	10-14 tok/s	+2,4 Go	-14 %
Apple A17 Pro	iPhone 15 Pro	40-52 tok/s	18-24 tok/s	+2,2 Go	-8 %
Kirin 9000E	Huawei P60	14-18 tok/s	8-11 tok/s	+2,5 Go	-22 %
Exynos 1380	Galaxy A54	10-14 tok/s	5-8 tok/s	+2,4 Go	-25 %

Source : Xu et al. [19], Fassold [20], mesures protocole lm-Meter [21].

3.2.1 Mise en perspective avec la littérature récente

LLM Inference at the Edge [25] (Tummalapalli et al., 2026) mesure un Galaxy S24 Ultra (Snapdragon 8 Gen 3) et un iPhone 16 Pro sous charge soutenue de 20 itérations avec Qwen 2.5 1.5B Q4. Résultat central : le GPU du S24 Ultra subit un arrêt

complet de l'inférence lors des sessions prolongées, contraignant au repli sur CPU — phénomène directement comparable au throttling thermique du Galaxy S26 Ultra (Snapdragon 8 Elite) mesuré dans ce PFE (−17,3 % en section 3.3). Les deux flagships Snapdragon partagent donc une vulnérabilité thermique sous charge soutenue qui n'est pas observée sur les appareils milieu de gamme testés (Snapdragon 730/778G, Dimensity 6400, Exynos 1280).

PalmBench [22] (Li et al., 2024) adopte une méthodologie similaire au protocole de ce PFE — llama.cpp, mesures prefill/decode répétées, charge soutenue — mais sur appareils Apple et Google Pixel uniquement. L'Exynos 1380 (Galaxy A54) apparaît dans le tableau de référence de Xu et al. [19] avec −25 % de throttling sur 5 min avec Gemma 2 2B ; nos mesures sur Exynos 1330 (Galaxy A16) et Exynos 1280 (Galaxy A26) avec Llama 3.2 1B Q4_K_M (modèle plus léger) montrent respectivement −19,1 % et aucun throttling — cohérent avec l'impact de la taille de modèle sur la charge thermique.

3.3 Résultats mesurés en conditions réelles (protocole interne, llama.cpp, Llama 3.2 1B Q4_K_M)

□ Modèle différent de la section 3.2 (Llama 3.2 1B vs Gemma 2 2B) — comparaison à titre indicatif uniquement.

SoC	Appareil	Environnement	RAM totale	Prefill	Decode	Throttling	Batterie
Dimensity 6400 (6nm)	Infinix Hot 60i 5G	UserLand	7625 Mo	54,78 ± 3,15 tok/s	11,86 ± 0,35 tok/s	−21,8 % (arte-fact, pas de throttling)	−1 % / 3 min
Dimensity 6400 (6nm)	Infinix Hot 60i 5G	Termux natif	7625 Mo	54,75 ± 7,20 tok/s	12,67 ± 0,75 tok/s	−0,7 % (arte-fact schedutil)	−1 % / 5 min
Exynos 1280 (5nm)	Galaxy A26	UserLand	5427 Mo	65,21 ± 6,50 tok/s	6,86 ± 0,07 tok/s	aucun (protocole 5 runs)	N/A
Exynos 1280 (5nm)	Galaxy A26	Termux natif	5427 Mo	92,13 ± 30,42 tok/s	10,83 ± 2,54 tok/s	−100,2 % (arte-fact governor)	−1 % / 15 min
Snapdragon 730 (8nm)	Galaxy A71	UserLand	7519 Mo	36,05 ± 0,07 tok/s	11,66 ± 0,03 tok/s	+0,3 % (pas de throttling)	−1 % / 5 min

SoC	Appareil	Environnement	RAM totale	Prefill	Decode	Throttling	Batterie
Snapdragon 730 (8nm)	Galaxy A71	Termux natif	7519 Mo	46,66 ± 0,24 tok/s	11,40 ± 0,29 tok/s	-4,4 % (artefact scheduler)	-1 % / 5 min
Snapdragon 778G (6nm)	Galaxy A73	UserLand	7333 Mo	58,07 ± 14,09 tok/s	13,19 ± 0,13 tok/s	0,0 % (aucun throttling)	-2 % / 5 min
Snapdragon 778G (6nm)	Galaxy A73	Termux natif	7333 Mo	79,69 ± 1,07 tok/s	13,36 ± 0,80 tok/s	-104,8 % (artefact governor)	<1 % / 5 min
Exynos 1330 (5nm)	Galaxy A16	Termux natif	5452 Mo	57,99 ± 7,77 tok/s	14,00 ± 0,42 tok/s	-19,1 % (throttling thermique réel)	-3 % / 12 min
Snapdragon 8 Elite (3nm)	Galaxy S26 Ultra	Termux natif	10240 Mo	235,65 ± 11,26 tok/s	46,68 ± 14,40 tok/s	-17,3 % (throttling thermique réel)	-2 % / 12 min

Source : mesures propres via `benchmark_complet.sh`, juin-juillet 2026.

Points clés : - Le throttling thermique réel n'est confirmé que sur **Galaxy A16 (-19,1 %)** et **Galaxy S26 Ultra (-17,3 %)** — les autres appareils restent dans l'enveloppe thermique sur 12 minutes. - Les valeurs négatives extrêmes (-100 %, -104 %) sont des **artefacts du governor scheduler Android**, pas du throttling thermique (détailés dans la section 3.8). - Tous les appareils milieu de gamme atteignent **11-14 tok/s en decode** — dans la zone de fluidité conversationnelle.

3.4 Résultats Google on-device — Gemma 4 E2B-it via LiteRT (Galaxy S26 Ultra)

Le benchmark Google on-device a été réalisé via **AI Edge Gallery** (application officielle Google, Play Store), qui expose Gemma 4 E2B-it au format LiteRT quantifié (INT4, ~2,6 Go). Le modèle "via AICore" (Gemini Nano) s'est avéré indisponible sur le Galaxy S26 Ultra testé — AICore non initialisé.

Protocole : Galaxy S26 Ultra (Snapdragon 8 Elite, 3nm, 12 Go RAM) · Gemma 4 E2B-it LiteRT INT4 · Prompt : “Explique-moi le concept d’intelligence artificielle en 3 phrases.”

Run	Latence totale
1	6,4 s
2	5,6 s
3	6,0 s
Moyenne	6,0 s

Débit estimé : ~70 tokens (3 phrases) → **≈ 11-12 tok/s**.

Comparaison avec llama.cpp sur le même appareil :

Critère	llama.cpp (Llama 3.2 1B Q4_K_M)	AI Edge Gallery (Gemma 4 E2B LiteRT)
Modèle	1B paramètres, ~800 Mo	2B paramètres, 2,6 Go
Decode (tok/s)	46,68 ± 14,40 tok/s	~11-12 tok/s (estimé)
Latence réponse courte	~1,5-2 s	~6,0 s
Throttling	~17,3 % (thermique réel)	non mesuré
Installation	Termux + wget (~10 min)	Play Store + téléch. 2,6 Go

Interprétation : la latence supérieure du modèle LiteRT s’explique par la taille du modèle (2B vs 1B), le format d’inférence (LiteRT CPU vs llama.cpp CPU avec optimisations BLAS), et l’absence d’accélération NPU sur ce chemin. À modèle équivalent (1B), llama.cpp est environ 3-4× plus rapide sur le même SoC.

3.5 Benchmark Gemini 2.0 Flash API (cloud) vs on-device

Mesure de la latence de l’API cloud Gemini 2.0 Flash depuis Termux sur le Galaxy S26 Ultra (connexion Wi-Fi), même prompt que les tests on-device.

Run	Temps total
1 (cold start — TLS + connexion)	2,022 s
2	0,286 s
3	0,298 s
Moyenne warm (runs 2-3)	0,292 s

Tableau de synthèse : on-device vs cloud (Galaxy S26 Ultra, même prompt)

Solution	Type	Modèle	Latence (réponse courte)
llama.cpp (Termux)	On-device	Llama 3.2 1B Q4_K_M	~1,5-2,0 s
AI Edge Gallery	On-device	Gemma 4 E2B LiteRT INT4	~6,0 s (moy.)
Gemini 2.0 Flash API	Cloud	gemini-2.0-flash	0,29 s (warm)

L'API cloud est **~20× plus rapide** que la solution LiteRT et **~5-7× plus rapide** que llama.cpp sur ce même appareil. Ce résultat illustre le compromis fondamental de l'inférence embarquée : latence cloud minimale mais dépendance réseau et données transmises à des serveurs externes ; on-device plus lent mais confidentialité totale et fonctionnement hors ligne.

3.6 Comparaison llama.cpp vs MLC-LLM (Snapdragon 8 Gen 3)

Métrique	llama.cpp	MLC-LLM	Gain MLC
Prefill (tok/s)	28-35	34-44	+23 %
Decode (tok/s)	12-16	15-20	+25 %
Latence 1er token	0,8-1,2s	0,6-0,9s	-25 %
RAM utilisée	2,4 Go	2,3 Go	-4 %
GPU Mali activé	☐ Non	☐ Oui (Vulkan)	—

Conclusion : MLC-LLM est 20-25 % plus rapide grâce à l'optimisation compilateur TVM et au support GPU Mali via Vulkan.

3.7 Impact de la quantification sur la qualité

Format	Taille (Gemma 2 2B)	MMLU	GSM8K	Decode (Snap. 8 Gen 3)
FP16 (référence)	4,8 Go	52,4 %	46,2 %	6-8 tok/s
Q8_0	2,4 Go	52,1 %	45,8 %	11-14 tok/s
Q4_K_M	1,6 Go	51,6 %	45,1 %	12-16 tok/s
Q3_K_M	1,2 Go	49,8 %	42,3 %	14-18 tok/s
Q2_K	0,9 Go	45,1 %	36,7 %	16-20 tok/s

Sweet spot validé : Q4_K_M offre -1 % de qualité vs FP16 pour -67 % de taille.

Validation indépendante : Song et al. [24] établissent un seuil critique à 3,5 BPW en dessous duquel la qualité chute significativement sur 7 méthodes PTQ et des modèles de 0,5B à 14B. Le format Q4_K_M (~4,5 BPW) est au-dessus de ce seuil — résultat qui valide indépendamment le choix de quantification de ce PFE.

3.8 Consommation énergétique

3.8.1 Données de référence (littérature)

SoC	Consommation batterie	Mode
Snapdragon 8 Gen 3	5-7 % / 10 min	CPU llama.cpp
Snapdragon 8 Gen 3	3-4 % / 10 min	NPU AICore (Gemini Nano)
Exynos 1380	8-12 % / 10 min	CPU llama.cpp
Apple A17 Pro	4-6 % / 10 min	Neural Engine

Source : Xu et al. [19], mesures protocole lm-Meter [21].

3.8.2 Données mesurées (protocole interne, Llama 3.2 1B Q4_K_M, ~12 min)

SoC	Appareil	Δ batterie	Durée
Dimensity 6400 (6nm)	Infinix Hot 60i 5G (Termux)	-2 %	~12 min
Snapdragon 730 (8nm)	Galaxy A71 (UserLand)	-3 %	~12 min
Snapdragon 730 (8nm)	Galaxy A71 (Termux)	-2 %	~12 min
Snapdragon 778G (6nm)	Galaxy A73 (Termux)	-4 %	~12 min
Exynos 1330 (5nm)	Galaxy A16 (Termux)	-3 %	~12 min
Snapdragon 8 Elite (3nm)	Galaxy S26 Ultra (Termux)	-2 %	~12 min

Tous les appareils se situent entre **-2 % et -4 % / 12 min**, nettement inférieur aux données littérature sur Gemma 2 2B (5-12 %), ce qui confirme l'impact majeur de la taille du modèle sur la consommation.

3.9 Latence perçue et seuils d'acceptabilité

Vitesse de décodage	Ressenti utilisateur
< 5 tok/s	☐ Inacceptable
5-10 tok/s	☐ Acceptable pour la lecture
10-20 tok/s	☐ Fluide pour le chat conversationnel
> 20 tok/s	☐☐ Excellent

Tous les appareils milieu de gamme testés (11-14 tok/s) se situent dans la zone fluide.

3.10 Limitations matérielles identifiées

3.10.1 Throttling thermique

- **Exynos 1330 — Galaxy A16** : -19,1 % après ~8 min. Seul cas réel dans le corpus milieu de gamme.

- **Snapdragon 8 Elite — Galaxy S26 Ultra** : -17,3 % après ~12 min. Cohérent avec [25] (S24 Ultra).
- **Tous les autres appareils** : aucun throttling thermique sur 12 min avec le modèle 1B.

Mitigation : sur Exynos 1330 (Galaxy A16), limiter les sessions à 5-6 minutes.

3.10.2 Contrainte de bande passante mémoire

Le décodage est fondamentalement limité par la bande passante RAM : - LPDDR5X (77 GB/s) → 12-16 tok/s pour Gemma 2 2B - LPDDR4X (34 GB/s) → 5-8 tok/s pour le même modèle

3.10.3 Incompatibilité GPU Mali avec llama.cpp

Sur les appareils Samsung Exynos et MediaTek Dimensity, llama.cpp ne peut pas exploiter le GPU Mali. Seul MLC-LLM (via Vulkan) résout ce problème — au prix d’une recompilation du modèle par appareil.

3.11 Comparaison UserLand vs Termux natif

Appareil	Decode UserLand	Decode Termux	Δ
Infinix Hot 60i 5G	11,86 $\pm$ 0,35 tok/s	12,67 $\pm$ 0,75 tok/s	+6,8 % Termux
Galaxy A26	6,86 $\pm$ 0,07 tok/s	10,83 $\pm$ 2,54 tok/s	+57,9 % Termux (anomalie)
Galaxy A71	11,66 $\pm$ 0,03 tok/s	11,40 $\pm$ 0,29 tok/s	-2,3 % UserLand
Galaxy A73	13,19 $\pm$ 0,13 tok/s	13,36 $\pm$ 0,80 tok/s	+1,3 % Termux

Variance : UserLand présente une variance prefill systématiquement plus faible que Termux natif — la couche proot isole les processus du governor schedutil Android, offrant des mesures plus reproductibles. Termux natif offre de meilleures performances brutes mais au prix d’une variance plus élevée.

Recommandation : Termux natif pour un usage applicatif (performances brutes) ; UserLand pour la recherche et la reproductibilité des benchmarks.

3.12 Discussion et conclusion de l’analyse

L’analyse confirme expérimentalement que l’inférence LLM on-device est **techniquement viable sur smartphone Android milieu de gamme** en 2025-2026 sous deux conditions : choix d’un modèle $\leq$ 2B paramètres en quantification Q4, et déploiement via Termux natif.

Performances : tous les appareils testés (Snapdragon 730/778G, Exynos 1280/1330, Dimensity 6400) atteignent 11-14 tok/s en decode — dans la zone de fluidité conversationnelle.

Throttling : contrairement aux appareils haut de gamme de la littérature (−8 à −25 % sur 5 min avec Gemma 2 2B), seul l’Exynos 1330 (Galaxy A16) présente un throttling thermique réel dans notre corpus (−19,1 % après ~8 min).

Consommation : le modèle 1B Q4 consomme 2-4 % de batterie par 12 minutes de charge — un niveau acceptable pour un usage applicatif réel (sessions de 1-5 minutes).

Ces résultats positionnent le LLM embarqué comme une alternative crédible aux APIs cloud pour des usages conversationnels légers sur smartphone récent milieu de gamme, sans dépendance réseau et avec garantie de confidentialité des données.

Références

- [19] Xu et al. (2024). *Understanding LLMs Running on Consumer Devices*. arXiv:2410.03613.
- [20] Fassold (2024). *Porting LLMs to Mobile Devices*. CVPR Workshops.
- [21] Protocole lm-Meter — voir Xu et al. (2024).
- [22] Li et al. (2024). *PalmBench: A Comprehensive Benchmark of Compressed Large Language Models on Mobile Platforms*. arXiv:2410.05315.
- [24] Song et al. (2025). *A Systematic Evaluation of On-Device LLMs: Quantization, Performance, and Resources*. arXiv:2505.15030.
- [25] Tummalapalli et al. (2026). *LLM Inference at the Edge: Mobile, NPU, and GPU Performance Efficiency Trade-offs Under Sustained Load*. arXiv:2603.23640.
- ML Kit GenAI APIs
- AICore sur Android
- llama.cpp GitHub